

Lista de Exercícios 02 — Prática com numpy e PyTorch

Regressão Linear: Equação Normal, MSE e RMSE

Disciplina: Inteligência Artificial

Prof. Gabriel Duarte

Objetivo

Implementar a **regressão linear** pela **equação normal** com **numpy** (solução fechada $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$) e avaliar o modelo com as métricas **MSE** (erro quadrático médio) e **RMSE** (raiz do erro quadrático médio), implementadas por você. As previsões e as métricas também devem ser calculadas com **tensores do PyTorch**, sem usar **sklearn** para ajustar o modelo (o **scikit-learn** entra apenas como *conferência* no fim). A entrega deve rodar em um **Google Colab** (CPU ou GPU) sobre um **dataset de regressão do UCI**, à sua escolha.

Configuração e Base de Dados

Onde rodar

Use o **Google Colab** (colab.research.google.com) ou um ambiente local com Python 3.10+ e PyTorch. Ao abrir o Colab, instale as dependências:

```
!pip install torch numpy scikit-learn matplotlib pandas
```

Verificação rápida

Para saber se o PyTorch está com a GPU disponível, rode:

```
import torch, numpy as np
print("torch:", torch.__version__, "| numpy:", np.__version__)
print("GPU disponível:", torch.cuda.is_available())
```

Se imprimir GPU disponível: `True`, você pode usar `device = "cuda"`. Caso contrário, use `device = "cpu"` (funciona do mesmo jeito, só mais lento).

A base de dados: escolha um dataset de regressão do UCI

O **UCI Machine Learning Repository** reúne bases públicas clássicas. Escolha **um** dos datasets abaixo (todos com links diretos de download). O exemplo de referência da lista é o **Auto MPG** (392 linhas válidas, 6 features numéricas, sem valores faltando após descartar 6 linhas com ?).

Dataset	Alvo (y)	Features		Download direto
Auto MPG	consumo (mpg)	6	numéricas	https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data
Wine Quality (tinto)	qualidade (0–10)	11	numéricas	https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv
Airfoil Self-Noise	nível de ruído	5	numéricas	https://archive.ics.uci.edu/ml/machine-learning-databases/00291/airfoil_self_noise.dat
Concrete Strength	resistência (MPa)	8	numéricas	https://archive.ics.uci.edu/ml/machine-learning-databases/concrete/compressive/Concrete_Data.xls
Energy Efficiency	carga de aquecimento	8	numéricas	https://archive.ics.uci.edu/ml/machine-learning-databases/00242/ENB2012_data.xls
Housing (Boston)	preço (mil US\$)	13	numéricas	https://archive.ics.uci.edu/ml/machine-learning-databases/housing/housing.data

Carregando o dataset no Colab

Dica: os arquivos terminam em `.data`, `.csv`, `.xls` ou `.xlsx`. Para os dois primeiros, um `read_csv` com separador *espaço* ou *vírgula* resolve:

```
import pandas as pd
# Auto MPG (colunas separadas por espaço, sem cabeçalho):
cols = ["mpg", "cylinders", "displacement", "horsepower",
        "weight", "acceleration", "model_year", "origin"]
df = pd.read_csv("auto-mpg.data", sep=" ", header=None, names=cols)
print(df.shape); print(df.head())
# Se houver "?" (missing), descarte as linhas:
tem_missing = df.astype(str).apply(
    lambda c: c.str.contains("\?", na=False)).any(axis=1)
df = df[~tem_missing]
X = df.drop(columns=["mpg", "origin"]).to_numpy(dtype=float)
y = df["mpg"].to_numpy(dtype=float)
print("X:", X.shape, "| y:", y.shape)
Para os arquivos .xls/.xlsx, use pd.read_excel("arquivo.xls") (pode precisar de !pip
install xlrd ou openpyxl).
```

Atenção

Todo o código que **implementa o modelo** (equação normal, previsão, MSE, RMSE) deve ser escrito **por você**, usando `numpy` e `torch` como *ferramentas de cálculo*. Usar `sklearn.linear_model.LinearRegression` para *ajustar* o modelo zera a nota do exercício correspondente; ele pode aparecer apenas na **Parte C**, como conferência.

Pré-processamento obrigatório

Lembra da aula: **padronize** as features (z-score) e **separe** treino/teste **antes** de ajustar. Padronize usando *estatísticas só do treino* e aplique as mesmas no teste:

```
from sklearn.model_selection import train_test_split
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
                                     random_state=42)
```

```
mu = Xtr.mean(axis=0); sd = Xtr.std(axis=0)
Xtr = (Xtr - mu) / sd; Xte = (Xte - mu) / sd
```

Por que padronizar? Deixa todas as features na mesma escala, o que torna a matriz $\mathbf{X}^\top \mathbf{X}$ bem condicionada (inversão estável) e permite **comparar os pesos** entre si — um peso grande (em módulo) passa a indicar uma feature importante.

1 Exercício 1 — Equação Normal com numpy

Você vai usar: `numpy` para álgebra linear (`np.linalg.inv` ou `np.linalg.solve`).

Como funciona (recapitulando)

A reta (hiperplano) $\hat{y} = \mathbf{w}^\top \mathbf{x}$ é a que minimiza o erro quadrático médio $J(\mathbf{w}) = \frac{1}{m} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$. Igualando a derivada a zero, o mínimo tem solução fechada: a **equação normal** $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

Tarefas

1. **Carregar e inspecionar.** Carregue o dataset escolhido, descarte valores faltantes e separe a matriz de features $\mathbf{X}$ do vetor alvo $\mathbf{y}$. Imprima `X.shape` e `y.shape` e comente quantas amostras sobraram.
2. **Split.** Separe 80% para treino e 20% para teste com `train_test_split(..., random_state=42)` (reprodutível).
3. **Padronização.** Padronize as features com z-score, usando média e desvio *do treino* e aplicando ao teste (ver caixa de pré-processamento).
4. **Matriz de design.** Adicione a **coluna de 1s** ao $\mathbf{X}$ de treino (para o intercepto w_0): use `np.hstack([np.ones((m,1)), Xtr])`.
5. **Equação normal.** Implemente $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ com `numpy`. Dica: `np.linalg.solve(XtX, Xty)` é numericamente mais estável que inverter; experimente as duas e compare.
6. **Previsões.** Com o $\mathbf{w}^*$ obtido, faça previsões no treino e no teste (adicionando a coluna de 1s também ao teste): $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}^*$. Mostre as previsões dos 5 primeiros pontos do teste ao lado dos valores reais.
7. **(Desafio) Interpretação dos pesos.** Identifique a feature de maior peso (em módulo) e interprete o sinal dela: no seu dataset, essa feature aumenta ou diminui o alvo quando cresce? Isso faz sentido no contexto do problema?

2 Exercício 2 — Avaliação com MSE e RMSE

Você vai usar: suas próprias funções de MSE e RMSE, primeiro com `numpy`, depois com **tensores do PyTorch**.

Como funciona (recapitulando)

Dado o vetor de erros $e_i = y_i - \hat{y}_i$, definimos:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n e_i^2, \quad \text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2}.$$

O MSE penaliza erros grandes (o quadrado), mas fica em *unidades ao quadrado* do alvo; o RMSE traz a medida de volta para a *unidade original* do problema — é o “erro típico” do modelo na mesma escala de y .

Tarefas

1. **MSE com numpy.** Implemente $\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$ com `numpy` (sem usar `sklearn.metrics`) e calcule no *treino* e no *teste* com as previsões do Exercício 1.
2. **RMSE com numpy.** Implemente o RMSE como a raiz quadrada do MSE e calcule nos mesmos conjuntos. Reporte em uma pequena tabela: conjunto (treino/teste) $\times$ (MSE, RMSE).
3. **Tensores do PyTorch.** Converta $\mathbf{X}$ (com coluna de 1s) e $\mathbf{y}$ de teste para `torch.tensor(..., dtype=torch.float32)` (escolha o `device`). Reproduza as previsões $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}^*$ em tensor e recalcule o MSE e o RMSE *só com operações de tensor* (`** 2`, `.mean()`, `torch.sqrt`).
4. **Igualdade.** Compare os valores de MSE/RMSE obtidos com `numpy` e com tensores: imprima a diferença máxima entre eles e comente se é desprezível.
5. **Conferência com sklearn.** Ajuste `LinearRegression` no mesmo treino padronizado e compare MSE e RMSE no teste com os seus valores. Comente a diferença encontrada (espera-se praticamente zero).
6. **(Desafio) Interpretação do RMSE.** Em palavras: o RMSE do seu teste representa o quê no contexto do problema? (Ex.: no Auto MPG, um RMSE de ≈ 3 mpg significa que o modelo erra, em média, cerca de 3 milhas por galão.)
7. **(Desafio) Sem padronização.** Refaz o Exercício 1 *sem* padronizar as features e compare os MSE/RMSE (treino e teste) com os da versão padronizada. Os erros mudam? E os pesos? Explique o que a padronização mudou na prática.

3 Exercício 3 — Previsão concreta e análise

Você vai usar: suas funções de previsão e métricas já implementadas.

Como funciona (recapitulando)

A regressão ajusta o modelo nos dados de treino; o teste mede a capacidade de *generalizar* (prever dados não vistos). Uma previsão concreta mostra o modelo funcionando em uma unidade do problema real.

Tarefas

1. **Previsão concreta.** Escolha a **primeira linha do teste**. Usando as estatísticas de padronização (μ, σ) *do treino*, padronize as features dessa linha, adicione a coluna de 1s e faça a previsão com o seu $\mathbf{w}^*$.
2. **Unidade original.** *Atenção:* só as **features** foram padronizadas — o alvo y permaneceu na unidade original (ex.: mpg). Logo a previsão $\hat{y}$ *já sai* na unidade do problema, sem despadronizar nada. Imprima o valor real, o valor previsto e o erro absoluto $|y - \hat{y}|$ dessa amostra. O que aconteceria se você padronizasse também o alvo antes de treinar? (Pense: o RMSE sairia em que unidade?)
3. **Erro típico.** Compare o erro absoluto dessa amostra com o RMSE do teste. A amostra errou mais ou menos que o “erro típico” do modelo?
4. **(Desafio) Feature mais influente.** Retome o peso de maior módulo do Exercício 1: aumente *apenas* essa feature em 1 desvio padrão (mantendo as demais na média), refaça a previsão e meça quantas unidades do alvo isso muda. Relacione com o peso encontrado.

4 Entrega e Critérios de Avaliação

Formato da entrega

Um **notebook do Colab** (.ipynb) bem comentado, com:

- a implementação da equação normal com numpy (Exercício 1);
- as funções de MSE e RMSE implementadas por você, em numpy e em tensores (Exercício 2);
- a previsão concreta com despadronização (Exercício 3);
- a tabela comparativa com **sklearn** como conferência;
- um resumo final com o dataset escolhido, o link de download e o MSE/RMSE obtidos.

Critérios de avaliação

Critério	Valor	Como é avaliado
Equação normal completa (numpy)	3,0	coluna de 1s, $\mathbf{w}^*$ correto, previsões no treino e teste
MSE e RMSE implementados (numpy e tensores)	3,0	fórmulas por você, treino e teste, igualdade numpy $\times$ tensor
Previsão concreta e análise	2,0	despadronização correta, comparação com o RMSE
Desafios (interpretação, sem padronizar)	1,0	análises corretas
Organização, comentários em PT-BR	1,0	código legível, células comentadas

Comparativo final (preencher)

Ao terminar, preencha mentalmente (ou em um comentário no final do notebook):

- MSE e RMSE deram *praticamente* os mesmos valores entre numpy, tensores e **sklearn**? Qual a maior diferença?
- O RMSE é maior ou menor que o MSE? Por quê (pense na unidade de cada um)?
- Qual feature do seu dataset mais influencia o alvo? O sinal do peso faz sentido?